DESENVOLVIMENTO WEB COM .NET

ANÁLISE E DESENVOLVIMENTO DE SISTEMAS

RELATÓRIO DE AULAS PRÁTICAS

Relatório 1 – Pipeline HTTP, middleware, roteamento, binding e escolha entre Minimal APIs e MVC

Nome: _____

RA: _____

Polo de matrícula: _____

Local da realização da Aula Prática: _____

Ano da postagem: 2026

Instituto de Ciências Exatas e Tecnologia	**Disciplina:** Desenvolvimento Web com .NET **Título da Aula:** Pipeline HTTP, middleware, roteamento, binding e escolha entre Minimal APIs e MVC	**Relatório 1**

Unidade 1 – Pipeline HTTP e endpoints

Objetivo: Construir a base do Portal de Tarefas, observar o percurso de uma requisição HTTP e implementar endpoints equivalentes com Minimal APIs e um controller MVC.

1. Identificação, ambiente e reprodução

Repositório: https://github.com/maiaralopes-dev/projetos (branch `ccr-836bb896-v5ejij`, pasta `PortalTarefas/`). Código desta unidade no commit `aa99393`. Projeto `PortalTarefas.Web` (alvo `net10.0`) na solução `PortalTarefas.slnx`, com `global.json` fixando o SDK 10.0.1xx. Para reproduzir: `dotnet run --project src/PortalTarefas.Web --launch-profile https` e executar o arquivo `PortalTarefas.Web.http` (ou `scripts/evidencias/atividade1.sh`).

Tabela 1 – Ambiente de desenvolvimento

Item	Versão efetivamente utilizada
SDK .NET / runtime	SDK 10.0.114 (CLI, Linux) · runtime ASP.NET Core 10.0.14 · C# 14
IDE	Visual Studio 2026 18.0 indicado no roteiro; o projeto foi produzido e verificado pela CLI `dotnet` (VS opcional)
Testes	xUnit 2.9.3 · Microsoft.AspNetCore.Mvc.Testing 10.0.14 · Vitest 5.1 · Testing Library 16

2. Resumo teórico aplicado ao projeto

Requisição e resposta HTTP. O cliente envia método, URI, cabeçalhos e, às vezes, corpo; o servidor devolve código de status, cabeçalhos e corpo. No ASP.NET Core o servidor Kestrel recebe a conexão, cria o `HttpContext` e o entrega ao pipeline; tudo o que a aplicação faz é ler `HttpContext.Request` e escrever em `HttpContext.Response`.

Middleware e ordem. Cada middleware recebe o contexto e decide se chama o próximo (`next`). O código antes de `await next()` roda na ida e o código depois roda na volta, como camadas de uma cebola; por isso a ordem de registro muda o comportamento. Exemplos no projeto: o middleware de correlação vem primeiro para que até respostas de erro tenham `X-Correlation-Id`; o `UseExceptionHandler` precisa estar *antes* dos endpoints para capturar suas exceções; o CORS fica entre `UseRouting` e os endpoints. **Encerrar antecipadamente** significa responder sem chamar `next`: o `UseStaticFiles` faz isso quando encontra o arquivo, e o endpoint `/ping` usa `.ShortCircuit()`, respondendo logo após o roteamento sem executar os middlewares seguintes.

Roteamento, endpoints e binding. `UseRouting` escolhe o endpoint pelo template (ex.: `/api/v1/tarefas/{id:int}`); o endpoint executa o handler. O binding converte partes da requisição em parâmetros: `id` vem da **rota** (com restrição `:int`), `situacao` e `busca` da **query**

string, `X-Cliente-Id` de um **cabeçalho** (`[FromHeader]`) e o cadastro do **corpo JSON** (`CriarTarefaRequest`).

Contrato previsível. Validação por Data Annotations (`AddValidation()`, novidade do .NET 10 para Minimal APIs), códigos coerentes (200, 201 + `Location`, 400, 404, 500) e erros no formato *Problem Details* permitem que o cliente trate falhas de forma automática. O `TratadorExcecoesGlobal` (`IExceptionHandler`) converte exceções em 500 sem mensagem, tipo ou *stack trace*, e erros de binding (`BadHttpRequestException`) em 400.

HTTPS, CORS e estáticos. `UseHttpsRedirection` responde 307 para HTTPS; a política CORS nomeada `frontend-dev` libera apenas `http://localhost:5173`, métodos e cabeçalhos listados; `UseStaticFiles` serve `wwwroot`.

Minimal APIs × MVC. Minimal APIs têm menos cerimônia, startup menor e filtros de endpoint; controllers MVC organizam muitas ações por classe, têm filtros de ação/resultado/exceção maduros, `[ApiController]` com validação automática e convenções conhecidas por equipes grandes. Testabilidade é equivalente com `WebApplicationFactory`.

3. Código-fonte e evidências

3.1 Ordem do pipeline implementado

1. `UseCorrelationId` – reaproveita/gera `X-Correlation-Id` e abre escopo de log;
2. `UseRegistroRequisicoes` – registra método, caminho, status e duração (sem query, cabeçalhos ou corpo);
3. `UseExceptionHandler + UseStatusCodePages` – Problem Details para exceções e respostas vazias 4xx/5xx;
4. `UseHsts` (fora de Development) e `UseHttpsRedirection`;
5. `UseStaticFiles → UseRouting → UseCors("frontend-dev")`;
6. endpoints: grupo Minimal API `/api/v1/tarefas`, diagnóstico Minimal API e `MapControllers()` (MVC).

O log resumido confirma o percurso “cebola”: entrada no middleware 1, entrada no 2, saída do 2 com status e duração, saída do 1 — inclusive na requisição com erro, em que o tratador registra a falha *entre* a entrada e a saída do middleware de log:

```
01:15:31 dbug CorrelationIdMiddleware      corr=teste-erro-0 [1-Correlação] entrada
/api/minimal/diagnostico/erro
01:15:31 info RequestLoggingMiddleware     corr=teste-erro-0 [2-Log] entrada GET
/api/minimal/diagnostico/erro
01:15:31 fail TratadorExcecoesGlobal       corr=teste-erro-0 Erro não tratado em GET
/api/minimal/diagnostico/erro System.InvalidOperationException: Falha simulada ao ace …
01:15:31 info RequestLoggingMiddleware     corr=teste-erro-0 [2-Log] saída GET
/api/minimal/diagnostico/erro -> 500 em 10.4 ms
01:15:31 dbug CorrelationIdMiddleware      corr=teste-erro-0 [1-Correlação] saída
/api/minimal/diagnostico/erro
```

3.2 Comparação Minimal API × MVC e decisão

Tabela 2 – Comparação das duas implementações equivalentes

Critério	Minimal API (`/api/minimal/diagnostico`)	Controller MVC (`/api/mvc/diagnostico`)
Organização	Grupos de rotas por recurso em métodos de extensão	Uma classe com várias ações e atributos
Dependências	Injetadas por parâmetro no handler	Injetadas no construtor do controller
Validação	`AddValidation()` (.NET 10) ou	`[ApiController]` + ModelState

	filtro de endpoint	automático
Filtros	`IEndpointFilter` por endpoint/grupo	Autorização, recurso, ação, exceção e resultado
Testes	`WebApplicationFactory`; handler pode ser método estático	`WebApplicationFactory` ou instância do controller
Contrato	Resposta idêntica (mesmo `DiagnosticoDto`) — verificado por teste	Idem

Decisão: a API do Portal (unidades 3 a 8) será feita em **Minimal APIs**, organizada em grupos (`MapGroup`) e métodos estáticos testáveis, porque o domínio é pequeno, os endpoints são finos e os recursos de .NET 10 (validação, `TypedResults`, OpenAPI nativo) cobrem as necessidades sem a cerimônia dos controllers. O **MVC** fica reservado à interface Razor (unidade 2), onde views, filtros de ação e Tag Helpers trazem ganho real. Se a equipe crescer e o número de endpoints exigir convenções fortes, migrar para controllers continua possível porque a lógica está nos serviços.

3.3 Requisições executadas (curl / arquivo .http)

Tabela 3 – Resumo das requisições (saída completa no Apêndice B)

Requisição	Status	Observação
`GET /api/v1/tarefas/1`	200	binding da rota; `X-Correlation-Id` na resposta
`GET /api/v1/tarefas/999`	404	Problem Details “Tarefa não encontrada.”
`GET /api/v1/tarefas/abc`	404	restrição `:int` – nem chega ao handler
`GET ...?situacao=Pendente&busca=relat` + `X-Cliente-Id`	200	filtros da query + cabeçalho ecoado em `X-Cliente-Atendido`
`GET ...?situacao=Arquivada`	400	enum inválido na query (antes da correção era 500)
`POST` corpo válido	201	`Location: /api/v1/tarefas/4`
`POST` título vazio / JSON malformado	400 / 400	`errors.Titulo` / “Requisição inválida.”
`GET /api/minimal/diagnostico/erro`	500	sem mensagem interna; com `correlationId`
`OPTIONS` origem `localhost:5173` / `evil.example`	204 / 204	com / **sem** `Access-Control-Allow-Origin`
`GET http://localhost:5080/...`	307	redirecionamento para HTTPS

4. Testes e validações obrigatórias

Tabela 4 – Testes obrigatórios (10 testes de integração, todos aprovados)

Caso exigido	Resultado obtido	Evidência
Parâmetro de rota válido e inválido	200 para `1`; 404 Problem Details para `999` e `abc`	Tab. 3; testes `Rota_*`
Filtro por query + cabeçalho opcional	Lista só `Pendente`; cabeçalho ecoado quando enviado e ausente quando omitido	`Filtro_query_e_cabecalho_opcional`
POST válido e inválido	201 com `Location`; 400 com erros por campo	`Post_valido_*`, `Post_invalido_*`
Exceção controlada sem dados sensíveis	500 genérico; o texto “db-interno” só aparece no log do servidor	`Excecao_vira_500_sem_detalhes_sensiveis`

CORS só para a origem autorizada	Preflight com cabeçalhos CORS só para `localhost:5173`	`Cors_somente_para_origem_autorizada`

5. Aprimoramentos e análise

- **Middleware reutilizável por classe** (`CorrelationIdMiddleware`, `RequestLoggingMiddleware`) registrado por extensões de `IApplicationBuilder` (`UseCorrelationId`, `UseRegistroRequisicoes`).
- **Identificador de correlação** devolvido em `X-Correlation-Id`, presente no escopo de log e no Problem Details (`correlationId`). Valores recebidos só são aceitos se curtos e alfanuméricos (evita injeção em logs).
- **Testes de integração** (`Atividade1PipelineTests`, 10 casos) com `WebApplicationFactory`, incluindo a equivalência de contrato entre Minimal API e MVC.
- **Falhas encontradas e corrigidas:** (1) enum inválido e JSON malformado geravam 500; o tratador passou a mapear `BadHttpRequestException` para 400; (2) `TypedResults.NotFound(ProblemDetails)` não passava pelo serviço de Problem Details (sem `correlationId`) e foi trocado por `TypedResults.Problem`.

Referências

MICROSOFT. **ASP.NET Core Middleware**. Microsoft Learn, 2025. Disponível em: https://learn.microsoft.com/aspnet/core/fundamentals/middleware/. Acesso em: 9 nov. 2026.

MICROSOFT. **Routing in ASP.NET Core**. Microsoft Learn, 2025. Disponível em: https://learn.microsoft.com/aspnet/core/fundamentals/routing. Acesso em: 9 nov. 2026.

MICROSOFT. **Parameter binding in Minimal API apps**. Microsoft Learn, 2025. Disponível em: https://learn.microsoft.com/aspnet/core/fundamentals/minimal-apis/parameter-binding. Acesso em: 9 nov. 2026.

MICROSOFT. **Handle errors in ASP.NET Core APIs**. Microsoft Learn, 2025. Disponível em: https://learn.microsoft.com/aspnet/core/fundamentals/error-handling-api. Acesso em: 9 nov. 2026.

MICROSOFT. **Enable Cross-Origin Requests (CORS) in ASP.NET Core**. Microsoft Learn, 2025. Disponível em: https://learn.microsoft.com/aspnet/core/security/cors. Acesso em: 9 nov. 2026.

MICROSOFT. **Choose between controller-based APIs and minimal APIs**. Microsoft Learn, 2025. Disponível em: https://learn.microsoft.com/aspnet/core/fundamentals/apis. Acesso em: 9 nov. 2026.

FIELDING, R.; NOTTINGHAM, M.; RESCHKE, J. **RFC 9110: HTTP Semantics**. IETF, 2022. Disponível em: https://www.rfc-editor.org/rfc/rfc9110. Acesso em: 9 nov. 2026.

NOTTINGHAM, M.; WILDE, E.; DALAL, S. **RFC 9457: Problem Details for HTTP APIs**. IETF, 2023. Disponível em: https://www.rfc-editor.org/rfc/rfc9457. Acesso em: 9 nov. 2026.

Apêndice A – Código-fonte integral

Arquivo: `src/PortalTarefas.Web/Program.cs` (commit `aa99393`)

```
using System.Text.Json.Serialization;
using PortalTarefas.Web.Endpoints;
using PortalTarefas.Web.Infraestrutura;
using PortalTarefas.Web.Servicos;

var builder = WebApplication.CreateBuilder(args);

// ---------- Serviços ----------
const string PoliticaCorsFrontend = "frontend-dev";
var origensPermitidas = builder.Configuration.GetSection("Cors:Origens").Get<string[]>() ?? [];

builder.Services.AddCors(o => o.AddPolicy(PoliticaCorsFrontend, p => p
    .WithOrigins(origensPermitidas)          // somente origens explícitas
    .WithMethods("GET", "POST", "PUT", "DELETE")
    .WithHeaders("Content-Type", "X-Cliente-Id", "X-Correlation-Id")
    .WithExposedHeaders("X-Correlation-Id", "Location")));

builder.Services.AddProblemDetailsComCorrelacao();
builder.Services.AddExceptionHandler<TratadorExcecoesGlobal>();
builder.Services.AddValidation();             // validação de Data Annotations em Minimal APIs (.NET 10)
builder.Services.AddControllers();
builder.Services.ConfigureHttpJsonOptions(o => o.SerializerOptions.Converters.Add(new
JsonStringEnumConverter()));
builder.Services.AddSingleton<RepositorioTarefasMemoria>();

var app = builder.Build();

// ---------- Pipeline (a ordem importa) ----------
app.UseCorrelationId();          // 1. gera/propaga X-Correlation-Id e abre escopo de log
app.UseRegistroRequisicoes();    // 2. mede método, caminho, status e duração (inclusive de erros)
app.UseExceptionHandler();       // 3. converte exceções em Problem Details 500
app.UseStatusCodePages();        // 4. respostas 4xx/5xx sem corpo viram Problem Details
if (!app.Environment.IsDevelopment())
    app.UseHsts();
app.UseHttpsRedirection();       // 5. redireciona HTTP -> HTTPS
app.UseStaticFiles();            // 6. arquivos de wwwroot (encerra a requisição se encontrar o arquivo)
app.UseRouting();                // 7. seleciona o endpoint
app.UseCors(PoliticaCorsFrontend); // 8. CORS entre roteamento e endpoints

app.MapTarefasEndpoints();       // 9. endpoints Minimal API
app.MapDiagnosticoEndpoints();
app.MapControllers();            //    endpoints MVC

app.Run();
```

Arquivo: `src/PortalTarefas.Web/Infraestrutura/CorrelationIdMiddleware.cs` (commit `aa99393`)

```
namespace PortalTarefas.Web.Infraestrutura;

/// <summary>
/// Garante que toda requisição tenha um identificador de correlação.
/// Reaproveita o cabeçalho X-Correlation-Id enviado pelo cliente (se for seguro)
/// ou gera um novo, devolvendo-o na resposta e abrindo um escopo de log.
/// </summary>
public sealed class CorrelationIdMiddleware(RequestDelegate next, ILogger<CorrelationIdMiddleware>
logger)
{
    public const string NomeCabecalho = "X-Correlation-Id";
    public const string ChaveItem = "CorrelationId";

    public async Task InvokeAsync(HttpContext context)
    {
        var correlationId = ObterOuGerar(context);
        context.Items[ChaveItem] = correlationId;
        context.TraceIdentifier = correlationId;

        // O cabeçalho é adicionado antes de a resposta começar a ser enviada.
        context.Response.OnStarting(() =>
        {
            context.Response.Headers[NomeCabecalho] = correlationId;
            return Task.CompletedTask;
        });

        using (logger.BeginScope("CorrelationId:{CorrelationId}", correlationId))
        {
            logger.LogDebug("[1-Correlação] entrada {Path}", context.Request.Path);
            await next(context);
            logger.LogDebug("[1-Correlação] saída {Path}", context.Request.Path);
        }
    }
```

```
    private static string ObterOuGerar(HttpContext context)
    {
        var recebido = context.Request.Headers[NomeCabecalho].ToString();
        // Aceita somente valores curtos e alfanuméricos para evitar injeção em logs.
        if (!string.IsNullOrWhiteSpace(recebido) && recebido.Length <= 64 &&
            recebido.All(c => char.IsLetterOrDigit(c) || c == '-'))
        {
            return recebido;
        }
        return Guid.NewGuid().ToString("N");
    }
}
```

Arquivo: `src/PortalTarefas.Web/Infraestrutura/RequestLoggingMiddleware.cs` (commit `aa99393`)

```
using System.Diagnostics;

namespace PortalTarefas.Web.Infraestrutura;

/// <summary>
/// Registra método, caminho, status e duração de cada requisição.
/// Não registra query string, cabeçalhos ou corpo, pois podem conter dados sensíveis.
/// </summary>
public sealed class RequestLoggingMiddleware(RequestDelegate next, ILogger<RequestLoggingMiddleware>
logger)
{
    public async Task InvokeAsync(HttpContext context)
    {
        var inicio = Stopwatch.GetTimestamp();
        logger.LogInformation("[2-Log] entrada {Metodo} {Caminho}", context.Request.Method,
context.Request.Path);
        try
        {
            await next(context);
        }
        finally
        {
            var duracao = Stopwatch.GetElapsedTime(inicio).TotalMilliseconds;
            logger.LogInformation("[2-Log] saída {Metodo} {Caminho} -> {Status} em {DuracaoMs:F1} ms",
                context.Request.Method, context.Request.Path, context.Response.StatusCode, duracao);
        }
    }
}
```

Arquivo: `src/PortalTarefas.Web/Infraestrutura/TratadorExcecoesGlobal.cs` (commit `aa99393`)

```
using Microsoft.AspNetCore.Diagnostics;
using Microsoft.AspNetCore.Mvc;

namespace PortalTarefas.Web.Infraestrutura;

/// <summary>
/// Converte exceções não tratadas em Problem Details (RFC 9457) sem expor
/// mensagem, stack trace ou tipo interno. O detalhe fica somente no log,
/// associado ao identificador de correlação.
/// </summary>
public sealed class TratadorExcecoesGlobal(IProblemDetailsService problemDetails,
ILogger<TratadorExcecoesGlobal> logger)
    : IExceptionHandler
{
    public async ValueTask<bool> TryHandleAsync(HttpContext context, Exception exception,
CancellationToken ct)
    {
        // Erros de binding (JSON malformado, enum inválido na query) são falhas do cliente: 400.
        if (exception is BadHttpRequestException bad)
        {
            logger.LogWarning("Requisição inválida em {Metodo} {Caminho}: {Motivo}",
                context.Request.Method, context.Request.Path, bad.Message);
            context.Response.StatusCode = bad.StatusCode;
            return await problemDetails.TryWriteAsync(new ProblemDetailsContext
            {
                HttpContext = context,
                ProblemDetails = new ProblemDetails
                {
                    Status = bad.StatusCode,
                    Title = "Requisição inválida.",
                    Detail = "Verifique o formato dos parâmetros e do corpo JSON."
                }
            });
        }

        logger.LogError(exception, "Erro não tratado em {Metodo} {Caminho}", context.Request.Method,
```

```
context.Request.Path);

        context.Response.StatusCode = StatusCodes.Status500InternalServerError;
        return await problemDetails.TryWriteAsync(new ProblemDetailsContext
        {
            HttpContext = context,
            Exception = exception,
            ProblemDetails = new ProblemDetails
            {
                Status = StatusCodes.Status500InternalServerError,
                Title = "Erro interno ao processar a requisição.",
                Type = "https://datatracker.ietf.org/doc/html/rfc9110#section-15.6.1",
                Detail = "Informe o identificador de correlação ao suporte."
            }
        });
    }
}
```

Arquivo: `src/PortalTarefas.Web/Infraestrutura/PipelineExtensions.cs` (commit `aa99393`)

```
namespace PortalTarefas.Web.Infraestrutura;

/// <summary>Extensões reutilizáveis para registrar os middlewares do projeto.</summary>
public static class PipelineExtensions
{
    public static IApplicationBuilder UseCorrelationId(this IApplicationBuilder app) =>
        app.UseMiddleware<CorrelationIdMiddleware>();

    public static IApplicationBuilder UseRegistroRequisicoes(this IApplicationBuilder app) =>
        app.UseMiddleware<RequestLoggingMiddleware>();

    /// <summary>Adiciona correlationId a todo Problem Details produzido pela aplicação.</summary>
    public static IServiceCollection AddProblemDetailsComCorrelacao(this IServiceCollection services) =>
        services.AddProblemDetails(o => o.CustomizeProblemDetails = ctx =>
        {
            if (ctx.HttpContext.Items.TryGetValue(CorrelationIdMiddleware.ChaveItem, out var id))
                ctx.ProblemDetails.Extensions["correlationId"] = id;
            ctx.ProblemDetails.Instance ??= $"{ctx.HttpContext.Request.Method}
{ctx.HttpContext.Request.Path}";
        });
}
```

Arquivo: `src/PortalTarefas.Web/Endpoints/TarefasEndpoints.cs` (commit `aa99393`)

```
using Microsoft.AspNetCore.Http.HttpResults;
using Microsoft.AspNetCore.Mvc;
using PortalTarefas.Web.Contratos;
using PortalTarefas.Web.Servicos;

namespace PortalTarefas.Web.Endpoints;

/// <summary>Endpoints de tarefas com Minimal APIs, demonstrando binding de quatro fontes.</summary>
public static class TarefasEndpoints
{
    public static RouteGroupBuilder MapTarefasEndpoints(this IEndpointRouteBuilder app)
    {
        var grupo = app.MapGroup("/api/v1/tarefas").WithTags("Tarefas");

        // Filtros pela query string (?situacao=&busca=) e cliente pelo cabeçalho X-Cliente-Id
(opcional).
        grupo.MapGet("/", (
                [FromQuery] SituacaoTarefa? situacao,
                [FromQuery] string? busca,
                [FromHeader(Name = "X-Cliente-Id")] string? clienteId,
                RepositorioTarefasMemoria repo,
                HttpResponse response) =>
            {
                if (!string.IsNullOrEmpty(clienteId))
                    response.Headers["X-Cliente-Atendido"] = clienteId;
                return TypedResults.Ok(repo.Listar(situacao, busca));
            })
            .WithName("ListarTarefas");

        // Id pela rota com restrição :int — "abc" nem chega ao handler (404 por rota).
        grupo.MapGet("/{id:int}", Results<Ok<TarefaDto>, ProblemHttpResult> (int id,
RepositorioTarefasMemoria repo) =>
                repo.Obter(id) is { } tarefa
                    ? TypedResults.Ok(tarefa)
                    : TypedResults.Problem(statusCode: 404, title: "Tarefa não encontrada.",
                        detail: $"Não existe tarefa com id {id}."))
            .WithName("ObterTarefa");

        // Corpo JSON validado automaticamente (AddValidation do .NET 10) — 400 em caso de erro.
```

```
        grupo.MapPost("/", Created<TarefaDto> (CriarTarefaRequest req, RepositorioTarefasMemoria repo) =>
            {
                var tarefa = repo.Adicionar(req.Titulo, req.Descricao, req.Situacao);
                return TypedResults.Created($"/api/v1/tarefas/{tarefa.Id}", tarefa);
            })
            .WithName("CriarTarefa");

        return grupo;
    }
}
```

Arquivo: `src/PortalTarefas.Web/Endpoints/DiagnosticoEndpoints.cs` (commit `aa99393`)

```
using Microsoft.AspNetCore.Mvc;
using PortalTarefas.Web.Contratos;
using PortalTarefas.Web.Infraestrutura;

namespace PortalTarefas.Web.Endpoints;

/// <summary>Endpoint diagnóstico em Minimal API (equivalente ao DiagnosticoController).</summary>
public static class DiagnosticoEndpoints
{
    public static IEndpointRouteBuilder MapDiagnosticoEndpoints(this IEndpointRouteBuilder app)
    {
        app.MapGet("/api/minimal/diagnostico", (
                [FromHeader(Name = "X-Cliente-Id")] string? clienteId,
                HttpContext ctx,
                IWebHostEnvironment env) =>
            TypedResults.Ok(new DiagnosticoDto("MinimalAPI", env.EnvironmentName,
                ctx.Items[CorrelationIdMiddleware.ChaveItem]?.ToString() ?? "", clienteId,
DateTimeOffset.UtcNow)))
            .WithTags("Diagnóstico");

        // Provoca uma exceção para demonstrar o tratamento centralizado.
        app.MapGet("/api/minimal/diagnostico/erro", string () =>
                throw new InvalidOperationException("Falha simulada ao acessar dependência interna db-
interno:1433"))
            .WithTags("Diagnóstico");

        // Encerramento antecipado: ShortCircuit executa o endpoint sem passar pelos middlewares
restantes.
        app.MapGet("/ping", () => "pong").ShortCircuit();

        return app;
    }
}
```

Arquivo: `src/PortalTarefas.Web/Controllers/DiagnosticoController.cs` (commit `aa99393`)

```
using Microsoft.AspNetCore.Mvc;
using PortalTarefas.Web.Contratos;
using PortalTarefas.Web.Infraestrutura;

namespace PortalTarefas.Web.Controllers;

/// <summary>Controller MVC equivalente ao endpoint diagnóstico em Minimal API.</summary>
[ApiController]
[Route("api/mvc/diagnostico")]
public sealed class DiagnosticoController(IWebHostEnvironment env) : ControllerBase
{
    [HttpGet]
    public ActionResult<DiagnosticoDto> Get([FromHeader(Name = "X-Cliente-Id")] string? clienteId) =>
        Ok(new DiagnosticoDto("MVC", env.EnvironmentName,
            HttpContext.Items[CorrelationIdMiddleware.ChaveItem]?.ToString() ?? "", clienteId,
DateTimeOffset.UtcNow));
}
```

Apêndice B – Evidências de execução

Saídas completas em `docs/evidencias/atividade1/` (`requisicoes.txt`, `log-servidor.txt`, `log-resumido.txt`). Trechos:

Rota inexistente, rota inválida e filtro com cabeçalho

```
### Parâmetro de rota inexistente
$ curl https://localhost:7080/api/v1/tarefas/999
HTTP/2 404
content-type: application/problem+json
x-correlation-id: 5b8199e18a16459c9d9acf9fefe74111
{
```

```
    "type": "https://tools.ietf.org/html/rfc9110#section-15.5.5",
    "title": "Tarefa não encontrada.",
    "status": 404,
    "detail": "Não existe tarefa com id 999.",
    "instance": "GET /api/v1/tarefas/999",
    "traceId": "00-c01af482631f87a1959a05addc3562f7-9b4c5a8720b19e85-00",
    "correlationId": "5b8199e18a16459c9d9acf9fefe74111"
}

### Parâmetro de rota inválido (não numérico)
$ curl https://localhost:7080/api/v1/tarefas/abc
HTTP/2 404
content-type: application/problem+json
x-correlation-id: 08ee4ca55ccb4b02b4c98490d860938c
{
    "type": "https://tools.ietf.org/html/rfc9110#section-15.5.5",
    "title": "Not Found",
    "status": 404,
    "instance": "GET /api/v1/tarefas/abc",
    "traceId": "00-6c9fb5361502c0cd7ea5def849933aef-e00b47fd004e568d-00",
    "correlationId": "08ee4ca55ccb4b02b4c98490d860938c"
}

### Filtro por query string + cabeçalho opcional
$ curl -H X-Cliente-Id: app-relatorio https://localhost:7080/api/v1/tarefas?situacao=Pendente&busca=relat
HTTP/2 200
content-type: application/json; charset=utf-8
x-cliente-atendido: app-relatorio
x-correlation-id: ac61f0c8e9804e0bb02b8e5765871706
[
    {
        "id": 3,
        "titulo": "Escrever relatório 1",
        "descricao": null,
```

POST válido, inválido, JSON malformado e exceção controlada

```
### POST com corpo válido
$ curl -X POST -H Content-Type: application/json -d {"titulo":"Estudar middleware","descricao":"Ordem do
pipeline","situacao":"EmAndamento"} https://localhost:7080/api/v1/tare …
HTTP/2 201
content-type: application/json; charset=utf-8
location: /api/v1/tarefas/4
x-correlation-id: 9d785f63a8d647159eb74f641f9c4149
{
    "id": 4,
    "titulo": "Estudar middleware",
    "descricao": "Ordem do pipeline",
    "situacao": "EmAndamento"
}

### POST com dados inválidos
$ curl -X POST -H Content-Type: application/json -d {"titulo":"","descricao":null}
https://localhost:7080/api/v1/tarefas
HTTP/2 400
content-type: application/problem+json
x-correlation-id: c48e3f51f8d54a0db2bfc4dba7ec5807
{
    "type": "https://tools.ietf.org/html/rfc9110#section-15.5.1",
    "title": "One or more validation errors occurred.",
    "status": 400,
    "instance": "POST /api/v1/tarefas",
    "errors": {
        "Titulo": [
            "O título é obrigatório."
        ]
    },
    "traceId": "00-40bd72dc563a213c91dd89c20953fa99-f6821b3d909ff78e-00",
    "correlationId": "c48e3f51f8d54a0db2bfc4dba7ec5807"
}

### POST com JSON malformado
$ curl -X POST -H Content-Type: application/json -d {"titulo": https://localhost:7080/api/v1/tarefas
HTTP/2 400
content-type: application/problem+json
cache-control: no-cache,no-store
x-correlation-id: 9dd45ca0fea6459db3bd6e7c5f3e5151
{
    "type": "https://tools.ietf.org/html/rfc9110#section-15.5.1",
```

Minimal API × MVC, CORS e HTTPS

```
### Diagnóstico Minimal API
```

```
$ curl -H X-Cliente-Id: cli-42 https://localhost:7080/api/minimal/diagnostico
HTTP/2 200
content-type: application/json; charset=utf-8
x-correlation-id: e36befb9641b42bca18cf4a5a2c485a7
{
    "abordagem": "MinimalAPI",
    "ambiente": "Development",
    "correlationId": "e36befb9641b42bca18cf4a5a2c485a7",
    "clienteId": "cli-42",
    "horario": "2026-11-09T01:15:31.8679985+00:00"
}

### Diagnóstico MVC (controller)
$ curl -H X-Cliente-Id: cli-42 https://localhost:7080/api/mvc/diagnostico
HTTP/2 200
content-type: application/json; charset=utf-8
x-correlation-id: 87513aa777cf4524b9674b199888fa1a
{
    "abordagem": "MVC",
    "ambiente": "Development",
    "correlationId": "87513aa777cf4524b9674b199888fa1a",
    "clienteId": "cli-42",
    "horario": "2026-11-09T01:15:31.9333165+00:00"
}

### CORS preflight - origem autorizada
$ curl -X OPTIONS -H Origin: http://localhost:5173 -H Access-Control-Request-Method: POST -H Access-
Control-Request-Headers: content-type https://localhost:7080/api/v1/tarefas
HTTP/2 204
access-control-allow-headers: Content-Type,X-Cliente-Id,X-Correlation-Id
access-control-allow-methods: GET,POST,PUT,DELETE
access-control-allow-origin: http://localhost:5173
x-correlation-id: 2a8ea659884f490c846813b0b36db821

### CORS preflight - origem NÃO autorizada
$ curl -X OPTIONS -H Origin: http://evil.example -H Access-Control-Request-Method: POST
https://localhost:7080/api/v1/tarefas
HTTP/2 204
x-correlation-id: 4031310a82da4ef79f41ec8daab2c1c9

### CORS GET simples - origem autorizada
```

Log resumido (ordem de entrada e saída dos middlewares)

```
01:15:31 dbug CorrelationIdMiddleware      corr=540801f42402 [1-Correlação] entrada /ping
01:15:31 info RequestLoggingMiddleware     corr=540801f42402 [2-Log] entrada GET /ping
01:15:31 info RequestLoggingMiddleware     corr=540801f42402 [2-Log] saída GET /ping -> 200 em 43.3 ms
01:15:31 dbug CorrelationIdMiddleware      corr=540801f42402 [1-Correlação] saída /ping
01:15:31 dbug CorrelationIdMiddleware      corr=743110bbf747 [1-Correlação] entrada /api/v1/tarefas/1
01:15:31 info RequestLoggingMiddleware     corr=743110bbf747 [2-Log] entrada GET /api/v1/tarefas/1
01:15:31 info RequestLoggingMiddleware     corr=743110bbf747 [2-Log] saída GET /api/v1/tarefas/1 -> 200 em 15.0
ms
01:15:31 dbug CorrelationIdMiddleware      corr=743110bbf747 [1-Correlação] saída /api/v1/tarefas/1
01:15:31 dbug CorrelationIdMiddleware      corr=5b8199e18a16 [1-Correlação] entrada /api/v1/tarefas/999
01:15:31 info RequestLoggingMiddleware     corr=5b8199e18a16 [2-Log] entrada GET /api/v1/tarefas/999
01:15:31 info RequestLoggingMiddleware     corr=5b8199e18a16 [2-Log] saída GET /api/v1/tarefas/999 -> 404 em 19.7
ms
01:15:31 dbug CorrelationIdMiddleware      corr=5b8199e18a16 [1-Correlação] saída /api/v1/tarefas/999
01:15:31 dbug CorrelationIdMiddleware      corr=08ee4ca55ccb [1-Correlação] entrada /api/v1/tarefas/abc
01:15:31 info RequestLoggingMiddleware     corr=08ee4ca55ccb [2-Log] entrada GET /api/v1/tarefas/abc
01:15:31 info RequestLoggingMiddleware     corr=08ee4ca55ccb [2-Log] saída GET /api/v1/tarefas/abc -> 404 em 2.4
ms
01:15:31 dbug CorrelationIdMiddleware      corr=08ee4ca55ccb [1-Correlação] saída /api/v1/tarefas/abc
01:15:31 dbug CorrelationIdMiddleware      corr=ac61f0c8e980 [1-Correlação] entrada /api/v1/tarefas
01:15:31 info RequestLoggingMiddleware     corr=ac61f0c8e980 [2-Log] entrada GET /api/v1/tarefas
01:15:31 info RequestLoggingMiddleware     corr=ac61f0c8e980 [2-Log] saída GET /api/v1/tarefas -> 200 em 13.4 ms
01:15:31 dbug CorrelationIdMiddleware      corr=ac61f0c8e980 [1-Correlação] saída /api/v1/tarefas
01:15:31 dbug CorrelationIdMiddleware      corr=c0abc3b91f0c [1-Correlação] entrada /api/v1/tarefas
01:15:31 info RequestLoggingMiddleware     corr=c0abc3b91f0c [2-Log] entrada GET /api/v1/tarefas
01:15:31 info RequestLoggingMiddleware     corr=c0abc3b91f0c [2-Log] saída GET /api/v1/tarefas -> 200 em 0.7 ms
01:15:31 dbug CorrelationIdMiddleware      corr=c0abc3b91f0c [1-Correlação] saída /api/v1/tarefas
01:15:31 dbug CorrelationIdMiddleware      corr=4004a02b097e [1-Correlação] entrada /api/v1/tarefas
01:15:31 info RequestLoggingMiddleware     corr=4004a02b097e [2-Log] entrada GET /api/v1/tarefas
01:15:31 warn TratadorExcecoesGlobal       corr=4004a02b097e Requisição inválida em GET /api/v1/tarefas: Failed
to bind parameter "Nullable<SituacaoTarefa> situacao" from …
01:15:31 info RequestLoggingMiddleware     corr=4004a02b097e [2-Log] saída GET /api/v1/tarefas -> 400 em 3.9 ms
01:15:31 dbug CorrelationIdMiddleware      corr=4004a02b097e [1-Correlação] saída /api/v1/tarefas
01:15:31 dbug CorrelationIdMiddleware      corr=9d785f63a8d6 [1-Correlação] entrada /api/v1/tarefas
01:15:31 info RequestLoggingMiddleware     corr=9d785f63a8d6 [2-Log] entrada POST /api/v1/tarefas
01:15:31 info RequestLoggingMiddleware     corr=9d785f63a8d6 [2-Log] saída POST /api/v1/tarefas -> 201 em 15.1 ms
01:15:31 dbug CorrelationIdMiddleware      corr=9d785f63a8d6 [1-Correlação] saída /api/v1/tarefas
01:15:31 dbug CorrelationIdMiddleware      corr=c48e3f51f8d5 [1-Correlação] entrada /api/v1/tarefas
```